

這是一份為你的 **C++ SFML** 物件導向踩地雷專案 量身打造的完整專案說明書。這份說明書非常適合用來放在 GitHub 的 README.md, 或是作為學校期末專案、個人作品集的繳交報告。

專案說明書: C++ SFML 物件導向踩地雷 (Minesweeper Pro)

本專案使用 **C++** 語言結合 **SFML (Simple and Fast Multimedia Library) 2.5/2.6** 圖形函式庫, 採用物件導向程式設計 (**OOP**) 架構, 重構並實作了一款具備現代踩地雷核心機制的完整遊戲。

1. 遊戲核心用例 (User Cases)

以下是玩家在遊玩此款踩地雷遊戲時的典型操作與對應的系統用例:

- 用例一: 選擇遊戲難度與重置
 - 玩家動作: 在遊戲任何時刻按下鍵盤的 1、2、3 鍵。
 - 系統反應: 即時銷毀舊視窗, 並根據選擇的難度 (初級 10×10 / 中級 16×16 / 高級 30×16) 動態重新分配記憶體, 拉伸視窗並重置所有計時器與狀態。
 - 用例二: 第一鍵絕對安全防護
 - 玩家動作: 新局開始, 滑鼠左鍵任意點擊地圖上的某一格。
 - 系統反應: 系統在此時才開始在後台灑下地雷, 並自動避開玩家點擊位置的周圍九宮格, 保證第一手絕對不會踩雷, 且會順利連鎖翻開大片空格。
 - 用例三: 插旗標記與保護機制
 - 玩家動作: 滑鼠右鍵點擊懷疑有雷的灰色格子。
 - 系統反應: 格子切換為紅旗圖案。此時該格會受到保護, 滑鼠左鍵無法誤點翻開, 再次點擊右鍵可取消插旗。
 - 用例四: 進階快速展開 (Chording)
 - 玩家動作: 當某個數字格 (例如 3) 周圍的紅旗數量已經點滿 3 面時, 玩家直接對著該數字格點擊左鍵。
 - 系統反應: 系統自動掃描周圍其餘的灰色格子並將它們同時點開, 大幅加快掃雷速度。若玩家插錯旗, 此動作會直接引爆地雷導致遊戲結束。
 - 用例五: 勝負判定與全圖揭曉
 - 判定失敗: 點擊到地雷, 系統鎖定輸入, 畫面上強制翻開所有隱藏的地雷, 並在畫面正中央疊加置中的紅色 "GAME OVER" 字樣。
 - 判定勝利: 當所有非地雷的安全格子都被成功翻開時, 遊戲鎖定輸入, 並在中央顯示黃色 "YOU WIN!" 字樣, 計時器停止。
-

2. 核心架構與各功能程式碼詳細說明

本專案的核心是 Minesweeper 類別，它將遊戲的「資料結構」、「遊戲邏輯」與「畫面渲染」緊密封裝。以下是各模組函式的深度解析：

A. 資料結構定義

C++

```
struct GridCell {  
    bool isMine = false;    // 是否為地雷  
    bool isOpened = false;  // 是否已翻開  
    bool isFlagged = false; // 是否已插旗  
    int neighborMines = 0;  // 周圍地雷數 (0~8)  
};
```

- 說明：將每個格子的複合屬性包裝成 GridCell 結構體。相較於傳統使用多個 2D 二進位陣列打架的寫法，這種高內聚的結構能大幅降低程式碼出錯率。

B. 動態難度重置 (resetGame)

C++

```
void resetGame(int r, int c, int mines) {  
    rows = r; cols = c; totalMines = mines;  
    firstClick = true; gameOver = false; gameWon = false; timerStarted = false; elapsedSeconds = 0;  
    if (hasFont) timerText.setString("000");  
  
    board.assign(rows, std::vector<GridCell>(cols));  
    window.create(sf::VideoMode(cols * W * (int)SCALE, rows * W * (int)SCALE + UI_HEIGHT),  
        "Minesweeper Pro");  
}
```

- 說明：此函式是動態地圖的核心。使用 board.assign() 重新分配儲存空間並初始化。同時，利用 window.create() 可以在不重啟程式的前提下，即時調整作業系統視窗的大小。

C. 避開首點安全區佈雷 (placeMines & calculateNumbers)

C++

```
bool inSafetyZone = (abs(r - firstR) <= 1 && abs(c - firstC) <= 1);
```

- 說明：在 placeMines 中，利用絕對值 abs 計算隨機座標與玩家第一手點擊座標的距離。若 \$X\$ 與 \$Y\$ 的距離皆小於等於 1，代表落於九宮格安全區內，程式會跳過並重新抽籤，實現第一鍵絕對安全。之後再遍歷地圖，利用雙層 \$-1\$ 到 \$+1\$ 的迴圈累加周圍地雷數填入 neighborMines。

D. 遞迴連鎖翻開 (openCell)

C++

```
void openCell(int r, int c) {  
    if (r < 0 || r >= rows || c < 0 || c >= cols || board[r][c].isOpened || board[r][c].isFlagged) return;  
    board[r][c].isOpened = true;  
  
    if (board[r][c].neighborMines == 0 && !board[r][c].isMine) {  
        for (int i = -1; i <= 1; i++) {  
            for (int j = -1; j <= 1; j++) {  
                openCell(r + i, c + j);  
            }  
        }  
    }  
}
```

- 說明：經典的 **Flood Fill** (洪水填充) 遞迴演算法。當點擊到數字為 0 的空格時，函式會自我呼叫 (遞迴) 擴展檢查周圍 8 格。開頭的 if 條件是嚴格的邊界與防禦邊界判定，能有效防止陣列越界當機 (Out of Bounds) 與無窮遞迴導致的記憶體堆疊溢位 (Stack Overflow)。

E. 快速展開/和弦功能 (quickReveal)

- 說明：此函式首先掃描點擊數字格周圍 8 個方向的 isFlagged 總數。若數量相符，則再次遍歷周圍。若遇到未翻開且未插旗的格子，會檢查 isMine。如果是雷則立刻回傳 hitMine = true 引爆遊戲；如果是安全格則呼叫 openCell 自動展開。

F. 事件分流管理 (processEvents & handleMouseClicked)

- 說明：
 - processEvents 負責從視窗隊列中提取事件，並過濾掉不必要的滑鼠雜訊，只有當滑鼠座標大於 UI_HEIGHT (即點在地圖區內) 時，才會將像素座標換算成矩陣索引 $row = (mousePos.y - UI_HEIGHT) / (W * SCALE)$ 。
 - 換算後的資料傳遞給 handleMouseClicked，依照左鍵、右鍵、已翻開、未翻開等 4 種複合狀態進行精準的邏輯分流。

G. 即時時鐘與動態字串格式化 (update)

C++

```
std::string timeStr = std::to_string(elapsedSeconds);  
if (elapsedSeconds < 10) timeStr = "00" + timeStr;  
else if (elapsedSeconds < 100) timeStr = "0" + timeStr;
```

- 說明：利用 clock.getElapsedTime().asSeconds() 獲取精確到 CPU 毫秒級的時間，轉為整數秒後，透過條件判斷在字串前方補 0，完美還原了傳統 Windows 踩地雷經典的「三位數紅色 LED 計時器」視覺效果。

H. 精靈圖座標映射與畫面繪製 (render)

C++

```
sprite.setTextureRect(sf::IntRect(tileX * W, tileY * W, W, W));  
sprite.setPosition(j * W * SCALE, i * W * SCALE + UI_HEIGHT);
```

- 說明：整個專案只載入一張高效率的素材圖。在 render 中，根據網格的布林狀態運算，即時算出圖案在素材圖上的 tileX 與 tileY 索引，利用 setTextureRect 進行動態裁剪。在繪製到螢幕上時，縱軸一律加上 UI_HEIGHT，成功在畫面上方切分出一個乾淨漂亮的文字 UI 資訊看板。

3. 專案技術亮點總結

1. 高內聚低耦合：主程式 main 縮減至僅存 3 行，所有邏輯職責切分明確（事件、更新、渲染分

離)。

2. 動態記憶體管理:不使用靜態陣列死板固定大小, 利用 `std::vector` 的動態特性配合 SFML 視窗重置, 達成不重啟遊戲即可流暢切換難度。
3. 防禦型程式設計:移除了會造成標準庫全局命名空間污染的 `using namespace std;`, 並加入 `NOMINMAX` 巨集, 徹底隔離了系統層與外部函式庫的底層衝突, 專案建置極其穩定。